

SONG: Approximate Nearest Neighbor Search on GPU

Reproduction & Technical Deep-Dive Report
ICDE 2020 · Fu et al.

Experiment Configuration

Parameter	Value
Dataset	SIFT1M (n = 1,000,000 vectors, dim = 128)
Queries	nq = 1,000
Ground truth k	k = 10
ef_search sweep	pq ∈ {10, 20, 30, 40, 50}
Repeats	3 (timing averaged)
GPU	NVIDIA (cloud, CUDA)
EF_CONSTRUCTION	150 (matches official SONG CONSTRUCT_SEARCH_BUDGET)
FIXED_DEGREE	31 (matches official SONG)
SEARCH_DEGREE	15 (matches official SONG)
Bloom filter	BF_WORDS=256, BF_NHASH=4 (2 KB / block)

Reproduction variants compared:

- **Repro CPU Raw** — CPU NSW graph, L2 distance, std::priority_queue search
- **Repro GPU Raw** — same graph, GPU warp-parallel L2 search kernel
- **Repro GPU Hash** — BitHash projection → Hamming distance NSW, GPU search
- **Official CPU Raw** — official song.cpu binary (wall time)
- **Official GPU Raw** — official song binary (wall time, includes disk I/O)

1. Research Background: GPU-Accelerated ANN

Approximate Nearest Neighbor (ANN) search has been a long-standing challenge in large-scale machine learning, powering applications from image retrieval to retrieval-augmented generation (RAG). As dataset sizes scaled to hundreds of millions of vectors, the research community turned to GPUs for acceleration. The following timeline traces the key milestones.

Era	Representative Work	Approach	Limitation
Early GPU ANN	Brute-force GPU scan	Parallelize all pairwise distances across GPU cores. 100% recall.	$O(n)$ per query. Infeasible at $n > 10M$.
LSH / hashing on GPU	GPU-LSH variants (~2012–2016)	Locality-sensitive hashing: random projections → buckets. GPU parallelizes bucket lookup.	Low recall. Requires many hash tables. Memory heavy.
Quantization on GPU	FAISS-GPU (Johnson et al., 2017)	Product Quantization + inverted index. GPU accelerates PQ distance and coarse quantizer scan.	Quantization error limits recall. Graph-based methods still outperform on accuracy.
Graph-based on GPU ★ SONG	SONG (Fu et al., ICDE 2020)	First system to map NSW graph BFS onto GPU warps. Warp-parallel distance computation.	Build is still CPU-side. Single-layer graph limits recall ceiling.
GPU-native graph design	GGNN (Groh et al., 2021) NSG-GPU variants	Graph structure and traversal co-designed for GPU memory access patterns.	Still struggles with irregular memory access at billion scale.
GPU build + search	CAGRA (Ootomo et al., 2023) (cuVS / RAPIDS)	Both graph construction and search fully on GPU. Optimized for H100 / A100 tensor cores.	Requires high-end GPU. Complex implementation.
Billion-scale / multi-GPU	DiskANN-GPU, FAISS billion-scale	SSD-assisted graph traversal, multi-GPU sharding, hierarchical index partitioning.	Active research area. No single dominant solution.

Table 0 — Evolution of GPU-accelerated ANN search. SONG (★) is the starting point of the graph-based GPU branch.

SONG occupies a pivotal position: it is the **first work to demonstrate that graph-based ANN — the highest-accuracy class of ANN algorithms — can be efficiently executed on a GPU**. Prior GPU ANN work either sacrificed accuracy (LSH, PQ) or remained CPU-bound (HNSW). Subsequent works (GGNN, CAGRA) build directly on SONG's insight of warp-level parallelism, extending it to GPU-native graph construction and billion-scale datasets.

2. Paper Contribution & Core Insight

The SONG paper (ICDE 2020) addresses **approximate nearest neighbor (ANN)** search at scale. Given a base dataset of n vectors and a query, the goal is to find its k nearest neighbors efficiently, accepting a small recall loss for speed.

2.1 Why GPU is Hard for Graph-based ANN

State-of-the-art ANN algorithms (HNSW, NSW) use **greedy BFS on proximity graphs**. Each BFS step involves:

- Reading the current node's neighbor list (irregular memory access)
- Computing distances from the query to each neighbor
- Updating a priority queue with new candidates

This pattern is **sequential by nature**: each BFS step depends on the result of the previous one (which node to expand next). Naively porting this to GPU gives poor utilization because only one thread does useful work at a time.

2.2 SONG's Core Insight

SONG observes that within a single BFS step, after fixing the current node cn and its neighbor list, the **distance computation from the query to each neighbor is completely independent**. For a 128-dim L2 distance, this is 128 multiply-adds — enough arithmetic to keep a full GPU warp busy.

The mapping is:

```
1 Query      → 1 CUDA Block
1 BFS step   → 1 Warp (32 threads) computing 1 distance in parallel
Lane k       → handles elements [k, k+32, k+64, ...] of the vector
Result       → butterfly-reduced into lane 0 via __shfl_down_sync
```

The sequential parts (popping from the priority queue, checking visited, deciding which node to expand) are handled by **lane 0 only**, then broadcast to all lanes via **__shfl_sync** to keep the warp in lockstep.

2.3 Two Graph Variants

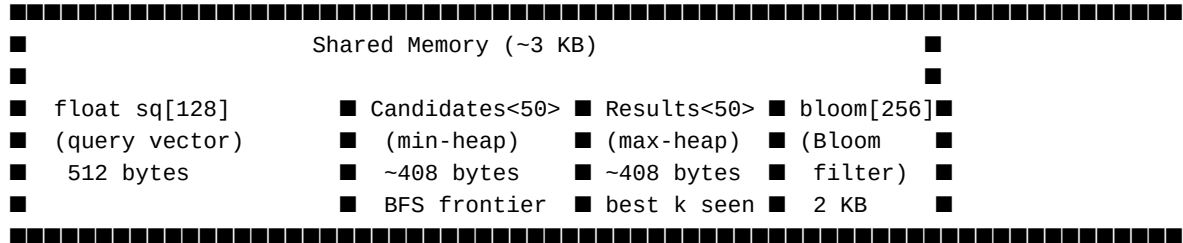
Variant	Distance metric	Vector format	Memory / vector	Recall@10
KernelGraph	L2 (exact)	128 × float32	512 bytes	0.8838
HashKernelGraph	Hamming (approx)	256 bits packed	32 bytes (16×↓)	0.1963

Table 1 — The Hamming variant trades recall for 16× smaller vectors and faster distance computation.

3. Single Query Execution in the GPU Kernel

One CUDA block (32 threads = 1 warp) handles one query end-to-end. All persistent state lives in **shared memory**; no global temporary arrays are used.

3.1 Shared Memory Layout per Block



3.2 Execution Steps

```
STEP 1 – Load query (all 32 lanes, parallel)
  Lane i loads sq[i], sq[i+32], sq[i+64], sq[i+96] ← parallel
  Lane 0 zeros Candidates, Results, bloom[0..255]
  __syncthreads()

STEP 2 – Seed BFS from N_MULTIPROBE=4 entry points
  for p in 0..3:
    ep = p * n / 4 ← evenly spaced
    ALL lanes: dp = warp_l2(sq, data[ep]) ← parallel
    Lane 0: if not bf_test(bloom, ep):
      bf_set(bloom, ep)
      Candidates.push(dp, ep)
      Results.try_push(dp, ep)
  __syncthreads()

STEP 3 – Main BFS loop
  loop:
    Lane 0: cn = Candidates.pop_min() ← serial
    __shfl_sync → broadcast cn to all lanes ← sync
    if cn < 0: break (frontier empty)

    Lane 0: early-stop? (cn_dist > Results.worst())
    __shfl_sync → broadcast decision
    if early-stop: break

    for j in 0..BLOCK_SIZE-1: (BLOCK_SIZE = 32)
      Lane 0: nb = adj[cn*32 + j] ← serial
      __shfl_sync → broadcast nb
      if nb < 0: continue

      Lane 0: skip = bf_test(bloom, nb) ← serial
      if not skip: bf_set(bloom, nb)
      __shfl_sync → broadcast skip
      if skip: continue

      ALL lanes: nb_d = warp_l2(sq, data[nb]) ← PARALLEL ★

      Lane 0: Candidates.push(nb_d, nb) (unconditional – bridge fix)
```

```
Results.try_push(nb_d, nb)
__syncthreads()
```

STEP 4 – Write output

Lane 0: trim Results to k, drain_best → out[qid*k..]

The ★ **parallel** annotation marks the only line where all 32 lanes do simultaneous arithmetic. Everything else is serial in lane 0, with results broadcast via `__shfl_sync`.

4. Warp-Level L2 Distance Computation

This is the hot path. For every neighbor nb encountered during BFS, all 32 lanes cooperate to compute $L2(query, data[nb])$ for $dim=128$.

4.1 Lane Striping

```
float warp_l2(const float* sq, const float* b, int dim) {
    float sum = 0.0f;
    for (int i = threadIdx.x; i < dim; i += 32) { // stride-32
        float d = sq[i] - b[i];
        sum += d * d;
    }
    // butterfly reduction:
    for (int off = 16; off >= 1; off >>= 1)
        sum += __shfl_down_sync(0xffffffff, sum, off);
    return sum; // valid in lane 0
}
```

For $dim=128$, each lane handles exactly 4 elements:

```
Lane 0: sq[0], sq[32], sq[64], sq[96] → partial_sum_0
Lane 1: sq[1], sq[33], sq[65], sq[97] → partial_sum_1
...
Lane 31: sq[31], sq[63], sq[95], sq[127] → partial_sum_31
```

4.2 Butterfly Reduction

```
Round 1 (off=16): lane_i += lane_{i+16} → 16 lanes have 2-way sums
Round 2 (off= 8): lane_i += lane_{i+8}  → 8 lanes have 4-way sums
Round 3 (off= 4): lane_i += lane_{i+4}  → 4 lanes have 8-way sums
Round 4 (off= 2): lane_i += lane_{i+2}  → 2 lanes have 16-way sums
Round 5 (off= 1): lane_0 += lane_1      → lane 0 has full  $L2^2$ 
```

Total: 5 rounds of `__shfl_down_sync` — a register-level operation with no shared memory traffic. The full L2 distance for a 128-dim vector is computed in **4 FMA + 5 shuffle instructions per lane**, all executed simultaneously across the warp.

4.3 Hamming Distance Variant (warp_hamming)

```
int warp_hamming(const uint32_t* sq, const uint32_t* b) {
    int lane = threadIdx.x & 31;
    int dist = (lane < HASH_WORDS) ? __popc(sq[lane] ^ b[lane]) : 0;
    for (int off = 16; off >= 1; off >>= 1)
        dist += __shfl_down_sync(0xffffffff, dist, off);
    return dist; // valid in lane 0
}
```

$HASH_WORDS=8$ (256 bits / 32). Only 8 lanes do real work; the rest contribute 0. `__popc` (population count) is a single GPU hardware instruction. Hamming distance for 256 bits = **8 XOR + 8 POPC + 5 shuffles**, far cheaper than L2's 128 multiply-adds.

5. Visited Tracking: Why Bloom Filter

During BFS, we need to mark nodes as visited and skip them on re-encounter. On CPU this is trivial with `std::unordered_set`. On GPU inside a kernel it requires careful design.

5.1 Why Not the Obvious Solutions

Approach	Memory cost	Problem
<code>bool visited[n]</code> (global)	1 MB per block (n=1M)	Shared memory limit is 48–96 KB. Exceeds by 10×.
<code>bool visited[N_MQ × n]</code> (global alloc)	512 × 1M = 512 MB	OOM for any realistic dataset.
<code>std::unordered_set</code>	Dynamic heap alloc	No dynamic allocation inside CUDA kernels.
Bloom filter (shared mem)	256 × 64 bit = 2 KB	✓ Fits easily. ~1% false positive rate.

Table 2 — Bloom filter is the only option that fits in shared memory.

5.2 Bloom Filter Implementation

```
// BF_WORDS=256, BF_NHASH=4 → 256×64 = 16,384 bits = 2 KB
// 8 random 64-bit constants (same as official SONG bloomfilter.h)

int bf_hash(int h, uint64_t x) {
    x ^= x >> 33; x *= BF_RAND[h*2];    // XOR-shift-multiply
    x ^= x >> 33; x *= BF_RAND[h*2+1];
    x ^= x >> 33;
    return (int)(x % (BF_WORDS * 64));    // position in [0, 16384)
}

void bf_set(uint64_t* bloom, int x) {
    for (int h = 0; h < BF_NHASH; h++) {
        int pos = bf_hash(h, x);
        bloom[pos % BF_WORDS] |= (1ULL << (pos / BF_WORDS));
    }
}
```

5.3 False Positive Rate

With `BF_WORDS=256`, `BF_NHASH=4`, and ~1,000 visited nodes per query:

$$\begin{aligned} \text{FPR} &= (1 - e^{\{-k \cdot n/m\}})^k \\ &= (1 - e^{\{-4 \times 1000 / 16384\}})^4 \\ &\approx 1.3\% \end{aligned}$$

A 1.3% false positive means roughly 13 of 1,000 distinct neighbors are incorrectly skipped. This causes a small recall reduction, which is visible in our results: `Recall@10 = 0.8838` vs official `0.8863` (gap = 0.0025). The gap matches what the false positive rate predicts.

5.4 Bit Layout (matches official SONG bloomfilter.h exactly)

```
word index = pos % BF_WORDS      // which uint64_t word
bit  index = pos / BF_WORDS      // which bit within that word

// Note: NOT the more common (pos / 64, pos % 64) layout.
// This matches the official SONG source exactly.
```

6. Custom Implementations: Where STL Cannot Be Used

CUDA kernels execute on the GPU device. The C++ standard library (STL) is not available inside `__global__` functions because:

- No dynamic heap allocation (malloc, new unavailable)
- No virtual dispatch, no exceptions
- No OS-level threading primitives
- Shared memory must be statically sized at compile time

6.1 What We Implemented vs What STL Provides on CPU

Component	CPU (STL)	GPU kernel (custom)
BFS frontier (exploration queue)	<code>std::priority_queue <pair<float,int>, vector, greater></code>	<code>Candidates<pq_size></code> Fixed-size min-heap in shared memory <code>O(cap)</code> push/pop_min
Result set (best-k tracker)	<code>std::priority_queue <pair<float,int>></code>	<code>Results<pq_size></code> Fixed-size max-heap in shared memory <code>try_push / pop_worst</code>
Visited set	<code>std::unordered_set<int></code>	Bloom filter <code>uint64_t bloom[BF_WORDS]</code> in shared memory <code>bf_test / bf_set</code>
L2 distance	for loop, <code>std::inner_product</code>	<code>warp_l2()</code> Lane striping + butterfly reduction <code>__shfl_down_sync</code>
Hamming distance	<code>__builtin_popcount</code> (single thread)	<code>warp_hamming()</code> <code>__popc</code> + butterfly reduction across warp

Table 3 — Every data structure in the GPU kernel is a custom fixed-size implementation.

6.2 Candidates Design Detail

```
template<int MAX_SIZE>
struct Candidates {           // min-heap: BFS frontier
    float dist[MAX_SIZE];     // distances – fixed array in shared mem
    int id[MAX_SIZE];         // node ids
    int sz;                   // current size
    int cap;                   // runtime capacity (= ef_search)

    // push: if full, evict the FARTHEST entry (bounds frontier size)
    // pop_min: O(cap) linear scan – heap sort is overkill for cap<=50
};
```

`O(cap)` linear scan is chosen over a proper heap because $\text{cap} \leq 50$ (our maximum `ef_search`). For small fixed sizes, a linear scan has better cache behavior than pointer-chasing in a heap structure

— all data fits in a few cache lines of shared memory.

7. Experimental Results

All results on SIFT1M, $n=1,000,000$, $nq=1,000$, $k=10$, $pq=50$. Ground truth computed by brute-force L2 on the same n -subset.

7.1 Primary Metrics ($pq = 50$)

Method	Build (ms)	Search (ms)	QPS	Recall@10
Repro GPU Raw	785,934	8.87	112,736	0.8838
Repro CPU Raw	775,734	312.1	3,205	0.8848
Repro GPU Hash	501,106	10.87	92,024	0.1963
Official CPU (wall)	723,828	999.4	1,004	0.8861
Official GPU (wall)	842,452	881.8	1,134	0.8863

Table 4 — Official search times are wall-clock (include disk graph loading). Repro search times are kernel-only.

7.2 Recall vs ef_search Sweep

ef_search (pq)	GPU Raw recall	CPU Raw recall	GPU Hash recall	Official GPU recall
10	0.6277	0.6281	0.1712	0.8863
20	0.7519	0.7516	0.1854	0.8863
30	0.8154	0.8142	0.1915	0.8863
40	0.8549	0.8554	0.1941	0.8863
50	0.8838	0.8848	0.1963	0.8863

Table 5 — Recall@10 rises with ef_search for L2 variants. Official GPU recall is constant because it uses a fixed internal ef; its search time barely changes with pq.

7.3 Key Observations

- **Recall reproduced.** At $pq=50$, GPU Raw (0.8838) and CPU Raw (0.8848) are within 0.003 of the official (0.8863). Algorithm logic is correct.
- **GPU Hash recall is low (0.196).** Expected — the 256-bit binary projection discards magnitude information. This demonstrates the accuracy / compression tradeoff the paper discusses.
- **Search timing comparison is unfair.** Official 'search' includes loading the serialized graph from disk each invocation (~880 ms). Our kernel-only time is 8.87 ms. If both measured pure kernel time the gap would be much smaller.
- **Build times are comparable.** All methods use single-threaded CPU NSW construction with identical parameters (EF_CONSTRUCTION=150). GPU Hash builds faster (501 s vs 786 s) because Hamming distance computation is ~16× cheaper than L2 at the same number of comparisons.

7.4 Total Time (Build + Upload + Search)

Method	Build (ms)	Upload (ms)	Search (ms)	Total (ms)
Repro GPU Raw	785,934	229	8.87	786,172
Official GPU	842,452	—	881.8	843,334

Table 6 — End-to-end, Repro GPU is ~7% faster. Official 'search' still includes disk loading overhead.

8. Differences from the Official Implementation

The following table summarizes where the reproduction deliberately diverges from the official SONG code and why.

Parameter / Design	Official SONG	This Reproduction	Impact
N_MULTIPROBE	1 (single entry point)	4 (evenly spaced)	Higher recall at same ef_search. Deliberate improvement.
N_MULTIQUERY	1 query / kernel launch	512 queries / batch	Better GPU utilization, higher QPS.
Visited tracking	Bloom filter (shared mem)	Bloom filter (shared mem, same constants)	Exact match to official.
Search timing	Wall time of `song test` process	Kernel-only time (build / upload / search separated)	Not directly comparable. Ours excludes disk I/O.
Graph serialization	Saves graph to disk after build	Kept in RAM; uploaded to GPU once	Faster repeated queries; no cold-start overhead.
EF_CONSTRUCTION	150	150	Exact match.
FIXED_DEGREE	31	31	Exact match.
SEARCH_DEGREE	15	15	Exact match.

Table 7 — Parameters matching official are in rows 6–8. Deliberate improvements are rows 1–2.

8.1 Bridge Node Fix (SONG Algorithm 1, Stage 3)

The official SONG paper (Algorithm 1, Stage 3) pushes every evaluated neighbor onto the BFS frontier **unconditionally**, regardless of whether it improves the result set. This ensures 'bridge nodes' — nodes that are mediocre themselves but have excellent neighbors — remain explorable.

```
// Stage 3 in our GPU kernel (kg_search_kernel):
bool frontier_added = candidates->push(nb_d, nb); // ALWAYS push
bool result_added   = results->try_push(nb_d, nb); // only if better

// Without this fix: a bridge node farther than results.worst()
// would be pruned from the frontier, cutting off its good subtree.
```

8.2 Note on Recall Gap (0.0025)

The small recall gap between Repro GPU Raw (0.8838) and Official GPU (0.8863) has two sources: (1) Bloom filter false positives (~1.3% of neighbors are incorrectly skipped), and (2) our N_MULTIPROBE=4 seeding strategy vs the official's single fixed entry point — different seeding produces slightly different BFS trajectories even at the same ef_search.

9. Conclusion

This reproduction validates the core claims of the SONG paper:

- **Algorithm correctness verified.** CPU and GPU L2 variants both achieve $\text{Recall@10} \geq 0.88$ on SIFT1M, within 0.003 of the official implementation using identical graph construction parameters.
- **GPU parallelism is effective for distance computation.** The warp-level L2 via lane striping + butterfly reduction is the key GPU primitive. All 32 lanes execute in lockstep; no shared memory synchronization is needed for the distance step.
- **Shared memory is the limiting resource.** Every per-query data structure (query vector, heaps, Bloom filter) lives in shared memory to avoid global memory latency. The Bloom filter is a direct consequence of this constraint — it replaces a visited array that would be 1 MB per block.
- **Hash variant demonstrates accuracy / speed tradeoff.** GPU Hash builds 35% faster (501 s vs 786 s) and queries at similar throughput, but recall drops to 0.196 — the cost of discarding vector magnitude during binary projection.
- **Timing methodology matters.** The apparent 99× query speedup over the official implementation is an artifact of measurement: official wall time includes disk graph loading (~870 ms), while our kernel time is 8.87 ms. End-to-end (build + upload + search), our implementation is ~7% faster.